Online, Reachability-Aware, Sampling-Based Motion Planning

Anonymous Authors

***Abstract*—Sampling-Based Model-Predictive Control (MPC) algorithms are a flexible class of controllers used for navigation on a wide range of robotic systems. Historically, such approaches have lacked hard safety guarantees, a shortcoming which we remedy in this work by computing guaranteed reachable-set overapproximations *online* with a fast, interval-based pipeline. We show that our method achieves similar performance to a state-of-the-art reachability-based planner without the need for the expensive pre-computation step, and can be scaled to systems that are infeasible using existing approaches. Finally, we demonstrate that our technique reduces safety violations by over 99% in a racing simulation and successfully controls a model racecar on real hardware experiments without crashes.**

***Index Terms*—Model-Predictive Control, Reachability Analysis, Safety Critical Systems**

I. Introduction

MODERN robotic systems, such as autonomous vehicles, warehouse robots, and drones, are expected to demonstrate a significant degree of independence in navigating through their environment. For many of these systems, a simple navigation failure may result in unacceptable danger or even harm to humans and other robots. Designing controllers for such *safety-critical* systems therefore requires additional care to ensure that catastrophic failures can be prevented, both in theory and in practice.

One popular framework used to tackle the problem of navigation and planning is Model-Predictive Control (MPC) [1], which uses a model of the system dynamics to predict future states and select inputs that induce desirable trajectories. This approach is notable for its flexibility; compared to classical approaches such as the linear-quadratic regulator, MPC places significantly less restrictive assumptions on the system dynamics and cost function. MPC algorithms have been applied to successfully control a wide variety of systems, including quadrotors [2], legged robots [3], and aquatic vehicles [4]. One popular approach of MPC is the Model-Predictive Path Integral (MPPI) algorithm [5], which controls a system by randomly sampling a set of control signals up to a finite horizon and assigning weights to each sample to minimize the Kullback-Leibler divergence between the sampled and optimal distributions.

Standard MPPI does not consider safety constraints, making it unsuitable for use with safety-critical systems. Prior work has proposed several methods to address this shortcoming. Some works incorporate safety into the cost function by penalizing trajectories that violate constraints or incur high levels of risk [6], [7], [8]. Others attempt to modify the distribution of sampled controls to balance safety and performance [9]. These techniques mitigate failure probabililty, but do not provide deterministic safety guarantees. Alternatively, some approaches append a safety *filter* (such as a control barrier function (CBF) [10]) to convert a possibly unsafe control into a safe one after sampling and weighting [11]. This idea is theoretically appealing, but designing a CBF for practical systems is often non-obvious, and filter interventions can affect performance in counter-intuitive ways [12]

Reachable-set over-approximation provides rigorous safety guarantees by bounding the set of states the system can reach at a given time [13], [14], [15]. Historically, such techniques have been too computationally expensive for online planning, and used instead for offline verification. A recent toolbox, `immrax` [16], was designed for fast and scalable reachability and addresses this issue with a combination of theoretical advancements [17], [18] and a state-of-the-art compuational implementation using the high-performance computing library JAX [19]. This, and other advances, have brought reachability-aware sampling planners within reach.

Recent work has started to take advantage of this possibility, most notably [20] which leverages Hamilton-Jacobi (HJ) reachability to guarantee safety under bounded disturbances. To do this, it pre-computes the HJ value function on a discretized grid over the state space, and interpolates between grid points at runtime to identify both the best safety-preserving control at any state and the last moment in time when such interventions can guarantee collision avoidance. The main weakness of such an approach is the need to solve a partial differential equation in advance, which becomes infeasible as system state dimension grows due to the curse of dimensionality [20] and, in addition, assumes that the system and environment are fully known beforehand.

In this work, we provide a similar guarantee to that of [20] while removing the expensive pre-computation. Through conservative, but fast, interval-based reachability using `immrax`, our algorithm performs all reachability calculations at runtime. The contributions of this work are threefold:

- A novel reachable-set aware sampling-based controller and its accompanying safety guarantee (Section III-C).
- A thorough empirical validation of the proposed controller both in simulation (Section IV-B) and on a 1/28th scale vehicle platform (Section IV-D).
- A two-way comparison to the state-of-the-art sampling-based planner [20] (Section IV-C), where our algorithm demonstrates similar results without any pre-computation.

A. Notation

For vectors $x, y \in \mathbb{R}^n$, we use the element-wise partial order $\leq$. Whenever $x \leq y$, let $[x, y] = \{z \in \mathbb{R}^n \mid x \leq z \leq y\}$, and $\mathbb{IR}^n$ be the set of all such intervals. The $\min$ and $\max$ operators on a finite collection of vectors are defined component-wise, and we abbreviate

$$\text{clip}\left(x; [\underline{x}, \overline{x}]\right) := \min\{\max\{x, \underline{x}\}, \overline{x}\}. \tag{1}$$

For any compact set $K \subseteq \mathbb{R}^n$, let $\mathrm{dist}(x, K)$ be the minimum distance between x and any point in K, and let ∂K be the boundary of K. For a continuous time function $p : [a, b] \to \mathbb{R}^n$, we use $p(c)$ to denote its value at $c \in [a, b]$, and $p(\cdot)$ to denote the entire function. Similarly, we use $p[c]$ and $p[\cdot]$ for discrete time signals. With a mild abuse of notation, we use the same symbol for both a discrete time function and the continuous time analogue formed by applying a zero-order hold between the function arguments.

II. Problem Statement

Consider a continuous time, non-linear system with state $\mathbf{x} \in \mathbb{R}^{d_\mathbf{x}}$, control input $\mathbf{u} \in \mathbb{R}^{d_\mathbf{u}}$, and disturbance input $\mathbf{w} \in \mathbb{R}^{d_\mathbf{w}}$ subject to the dynamics

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, \mathbf{w}). \tag{2}$$

The disturbance $\mathbf{w}$ is used to explicitly consider possible differences between an idealized, mathematical model and the real-world system. For a given time horizon T, input *signals* $\mathbf{u}(\cdot) \in L\left([0, T]; \mathbb{R}^{d_\mathbf{u}}\right)$ and $\mathbf{w}(\cdot) \in L\left([0, T]; \mathbb{R}^{d_\mathbf{w}}\right)$ are Lebesgue measurable functions from any time instant before that horizon to a particular input value.

Assumption 1. The vector field f satisfies the necessary regularity conditions such that the solutions to (2) exist and are unique on the time interval $[0, T]$ for all initial conditions $\mathbf{x}_{\text{init}} \in \mathbb{R}^{d_\mathbf{x}}$ and input signals $\mathbf{u}(\cdot)$, $\mathbf{w}(\cdot)$. We denote this solution by $\phi(t; \mathbf{x}_{\text{init}}; \mathbf{u}(\cdot), \mathbf{w}(\cdot))$.

Additionally, as we treat the disturbances as adversarial in nature, we require them to be bounded.

Assumption 2. There exists an interval $[\underline{\mathbf{w}}, \overline{\mathbf{w}}]$ such that $\mathbf{w}(t) \in [\underline{\mathbf{w}}, \overline{\mathbf{w}}]$ for all $t \in [0, T]$.

Our design goal is twofold and consists of both safety and performance components. To model safety, we use a function $h : \mathbb{R}^{d_\mathbf{x}} \to \mathbb{R}$ encoding a set of safe states

$$\mathcal{S}_\mathbf{x} := \{\mathbf{x} \mid h(\mathbf{x}) \geq 0\}. \tag{3}$$

This framework is particularly suited for e.g. collision avoidance, where states with $h(\mathbf{x}) \geq 0$ are not in collision and states with $h(\mathbf{x}) < 0$ are. At any $\mathbf{x}_{\text{init}}$, safe control signals are those that produce state trajectories contained in $\mathcal{S}_\mathbf{x}$ for the next T time units under any realization of $\mathbf{w}$:

$$\mathcal{S}_\mathbf{u}^T(\mathbf{x}_{\text{init}}) := \Bigg\{ \mathbf{u}(\cdot) \mid \inf_{\mathbf{w}(\cdot), t \in [0,T]} h(\phi(t; \mathbf{x}_{\text{init}}; \mathbf{u}(\cdot), \mathbf{w}(\cdot))) \geq 0 \Bigg\}. \tag{4}$$

Our primary design goal is to construct a safe control.

Problem 1. Given the current system state $\mathbf{x}_{\text{init}}$, identify a safe control signal $\mathbf{u}(\cdot) \in \mathcal{S}_\mathbf{u}^T(\mathbf{x}_{\text{init}})$, or verify that $\mathcal{S}_\mathbf{u}^T(\mathbf{x}_{\text{init}})$ is empty.

In addition to satisfying safety constraints, we often wish to design controls that achieve some objective. We model such objectives with a cost functional $J : L\left([0, T]; \mathbb{R}^{d_\mathbf{x}}\right) \times L\left([0, T]; \mathbb{R}^{d_\mathbf{u}}\right) \to \mathbb{R}$ that indicates which states and control actions are desirable.

Problem 2. Given the current system state $\mathbf{x}_{\text{init}}$, select an optimal control from the set of safe controls. That is, find

$$\mathbf{u}^*(\cdot) \in \underset{\mathbf{u}(\cdot) \in \mathcal{S}_\mathbf{u}^T(\mathbf{x}_{\text{init}})}{\arg\min} \; J(\phi(\cdot; \mathbf{x}_{\text{init}}; \mathbf{u}(\cdot), 0(\cdot)), \mathbf{u}(\cdot)). \tag{5}$$

In the next section, we present our solution to both of these problems.

III. Methodology

This section presents our theoretical contributions, which are organized as follows. Section III-A summarizes the established reachability technique used in our work, and Section III-B introduces a sampling-based motion planner. These ideas are combined in Section III-C, which contains both our proposed controller (Algorithm 2) and its corresponding safety guarantee (Theorem 1). Finally, Section III-D describes remaining opportunities for future work to better address Problems 1–2.

A. Interval Reachability

We leverage an interval-based reachability method developed in prior work [21]; it is summarized here for convenience. This method uses an *embedding system* whose trajectories define a time-varying interval set that is guaranteed to contain the trajectories of the original system, subject to bounded disturbances. Because of the simplicity of intervals, these bounds can be computed efficiently, at approximately 2× the cost of a trajectory of the original system.

The primary technique used to over-approximate the effect of disturbances on system trajectories is that of an inclusion function from interval arithmetic [22].

Definition 1 (Inclusion Function)**.** Let $f : \mathbb{R}^m \to \mathbb{R}^n$. We say $\left[\underline{f}, \overline{f}\right] : \mathbb{IR}^m \to \mathbb{IR}^n$ is an *inclusion function* for f if for any $x \in [\underline{x}, \overline{x}]$, $f(x) \in \left[\underline{f}, \overline{f}\right]([\underline{x}, \overline{x}])$. This notation has three components; a lower bound $\underline{f} : \mathbb{IR}^m \to \mathbb{R}^n$, an upper bound $\overline{f} : \mathbb{IR}^m \to \mathbb{R}^n$, and a combined $\left[\underline{f}, \overline{f}\right] : \mathbb{IR}^m \to \mathbb{IR}^n$ which outputs the interval between the lower and upper bounds.

Definition 1 generalizes to functions with multiple inputs naturally, requiring containment of the output for any combination of inputs within the specified intervals. Importantly, inclusion functions are not required to tightly contain their outputs (they may be conservative), nor unique.

As intuition may suggest, it is possible to over-approximate the trajectories of (2) by simply replacing f with one of its inclusion functions. However, a low-cost additional step can be used to greatly reduce the conservatism of such approximations by leveraging the structure of the continuous time dynamics. Geometrically, if any trajectory were to exit an interval $[\underline{\mathbf{x}}, \overline{\mathbf{x}}]$, it must first pass through one of the faces of $[\underline{\mathbf{x}}, \overline{\mathbf{x}}]$. We introduce the following notation to describe these faces:

$$\underline{\mathcal{B}_k}([\underline{\mathbf{x}}, \overline{\mathbf{x}}]) := \{\mathbf{x} \mid \mathbf{x} \in [\underline{\mathbf{x}}, \overline{\mathbf{x}}] \wedge \mathbf{x}_k = \underline{\mathbf{x}}_k\}, \tag{6}$$

$$\overline{\mathcal{B}_k}([\underline{\mathbf{x}}, \overline{\mathbf{x}}]) := \{\mathbf{x} \mid \mathbf{x} \in [\underline{\mathbf{x}}, \overline{\mathbf{x}}] \wedge \mathbf{x}_k = \overline{\mathbf{x}}_k\}. \tag{7}$$

Then, instead of over-approximating the expansion due to f over the entire state interval, we need only consider how quickly states that lie on a face of $[\underline{\mathbf{x}}, \overline{\mathbf{x}}]$ are expanding.

Definition 2 (Embedding System)**.** Consider the dynamical system (2). Let $\left[\underline{f}, \overline{f}\right]$ be an inclusion function for f. The corresponding *embedding system* has state $[\underline{\mathbf{x}}, \overline{\mathbf{x}}]$ with dynamics given by

$$\begin{bmatrix} \dot{\underline{\mathbf{x}}}_k \\ \dot{\overline{\mathbf{x}}}_k \end{bmatrix} = \begin{bmatrix} \underline{f}_k \left(\underline{\mathcal{B}}_k \left([\underline{\mathbf{x}}, \overline{\mathbf{x}}] \right), [\mathbf{u}, \mathbf{u}], [\underline{\mathbf{w}}, \overline{\mathbf{w}}] \right) \\ \overline{f}_k \left(\overline{\mathcal{B}}_k \left([\underline{\mathbf{x}}, \overline{\mathbf{x}}] \right), [\mathbf{u}, \mathbf{u}], [\underline{\mathbf{w}}, \overline{\mathbf{w}}] \right) \end{bmatrix}, \tag{8}$$

for all $k \in \{1, \ldots, d_{\mathbf{x}}\}$.

Note that each face of an interval is itself an interval set, so (8) is well-defined. Let $\left[\underline{\Phi}, \overline{\Phi}\right](t; \mathbf{x}_{\text{init}}; \mathbf{u}(\cdot))$ denote the solution of (8) from the initial condition $[\mathbf{x}_{\text{init}}, \mathbf{x}_{\text{init}}]$, where $\underline{\Phi}$ and $\overline{\Phi}$ individually give the lower and upper bounds of the interval trajectory. We have suppressed the dependence on error bounds $[\underline{\mathbf{w}}, \overline{\mathbf{w}}]$ in the notation to emphasize the fact that such trajectories are independent of any *realization* of error signal within those bounds. As desired, the time varying interval $\left[\underline{\Phi}, \overline{\Phi}\right]$ is guaranteed to contain trajectories of the original system under bounded disturbances.

Proposition 1 ([21, Theorem 1])**.** For any initial state $\mathbf{x}_{\text{init}}$, control signal $\mathbf{u}(\cdot) \in L\left([0, T]; \mathbb{R}^{d_{\mathbf{u}}}\right)$, and disturbance signal $\mathbf{w}(\cdot) \in L\left([0, T]; \mathbb{R}^{d_{\mathbf{w}}}\right)$ satisfying Assumption 2, we have, for all $t \in [0, T]$,

$$\phi(t; \mathbf{x}_{\text{init}}; \mathbf{u}(\cdot), \mathbf{w}(\cdot)) \in \left[\underline{\Phi}, \overline{\Phi}\right](t; \mathbf{x}_{\text{init}}; \mathbf{u}(\cdot)). \tag{9}$$

Remark 1. Application of Proposition 1 requires the computation of an inclusion function $\left[\underline{f}, \overline{f}\right]$ for the system dynamics. The software toolbox `immrax` [16] can automatically compute such a $\left[\underline{f}, \overline{f}\right]$ for any dynamics that are *factorable* into compositions of operations in a library of primitive mathematical functions. In Section IV, we use `immrax` to construct $\left[\underline{f}, \overline{f}\right]$ for a 6D dynamic bicycle vehicle model with integrated engine and tire slip models. We also demonstrate heuristics that can be used as computationally speedy alternatives for operations that are expensive to extend to intervals.

B. Trajectory-Based Motion Planning

Our algorithm follows a receding-horizon, sampling-based paradigm inspired by those used in prior work [5], [11], [20]. As a prelude to our proposed, reachability-aware planner, we now present a simplified approach. This section serves to establish notation and provide a benchmark comparison (see Section IV-B) that isolates the benefits of reachability analysis.

First, N candidate control signals are sampled from a Gaussian distribution centered around a reference $\mathbf{u}^*[\cdot]$. We consider piecewise-constant policies with uniform time discretization $\Delta_{\mathbf{u}} t$ and M segments, for a prediction horizon of $T = M\Delta_{\mathbf{u}} t$. After sampling, the controls are smoothed and clipped to enable low-level tracking. For precise implementation details, please see the project github repository.

Once a batch of controls is sampled, the system dynamics (2) are integrated for each sample (assuming zero disturbance), yielding a set of nominal state trajectories. Each nominal trajectory is checked for predicted safety violations and the remaining trajectories are evaluated using the cost functional:

$$\hat{\mathcal{S}}_{\mathcal{I}}^T(\mathbf{x}_{\text{init}}) := \Big\{ i \in \{1, \ldots, N\} \mid \inf_{t \in [0,T]} h\left(\phi\left(t; \mathbf{x}_{\text{init}}; \mathbf{u}^{(i)}[\cdot], 0(\cdot)\right)\right) \geq 0 \Big\} \tag{10}$$

$$i^* := \underset{i \in \hat{\mathcal{S}}_{\mathcal{I}}^T(\mathbf{x}_{\text{init}})}{\arg\min}\, J\left(\phi\left(t; \mathbf{x}_{\text{init}}; \mathbf{u}^{(i)}[\cdot], 0(\cdot)\right), \mathbf{u}^{(i)}[\cdot]\right) \tag{11}$$

Finally, the control differences corresponding to the optimal sampled control $\Delta\mathbf{u}^{(i^*)}[\cdot]$ are saved and used as the reference to "warm-start" the next control loop. The full strategy is given in Algorithm 1.

Algorithm 1 Sampling-Based Planner (Trajectories)

Require: Reference control sequence $\mathbf{u}^*[\cdot]$
function SAMPLE($\mathbf{u}^*[\cdot]$)
  $\mathbf{u}^*[\cdot] \leftarrow \begin{bmatrix} \mathbf{u}^*[2:M] & 0 \end{bmatrix}$ ▷ *progress time*
  $\mathbf{u}^{(1:N)}[\cdot] \leftarrow$ sample controls around $\mathbf{u}^*[\cdot]$
  return $\mathbf{u}^{(1:N)}[\cdot]$
function ROLLOUT($\mathbf{u}[\cdot]$, $\mathbf{x}_{\text{init}}$)
  return $\phi(\cdot; \mathbf{x}_{\text{init}}; \mathbf{u}[\cdot], 0(\cdot))$ by Euler integration
function SELECTBEST($\mathbf{x}^{(1:N)}(\cdot), \mathbf{u}^{(1:N)}[\cdot]$)
  $\hat{\mathcal{S}}_{\mathcal{I}}^T(\mathbf{x}_{\text{init}}) \leftarrow$ predicted safe samples from (10)
  return Optimal sample index as in (11)

$\mathbf{u}^{(1:N)}[\cdot] \leftarrow$ SAMPLE($\mathbf{u}^*[\cdot]$)
$\mathbf{x}^{(1:N)}(\cdot) \leftarrow$ ROLLOUT($\mathbf{u}^{(i)}[\cdot], \mathbf{x}_{\text{init}}$) for $\mathbf{u}^{(i)}[\cdot] \in \mathbf{u}^{(1:N)}[\cdot]$
$i^* \leftarrow$ SELECTBEST$\left(\mathbf{x}^{(1:N)}(\cdot), \mathbf{u}^{(1:N)}[\cdot]\right)$
Apply $\mathbf{u}^{(i^*)}[1]$, save $\mathbf{u}^{(i^*)}[\cdot]$

C. Reachability-Aware Motion Planning

We now present our main theoretical contribution: a generalization of Algorithm 1 that leverages the reachable sets produced by Proposition 1 to robustly guarantee safety in the presence of disturbances.

Instead of integrating the dynamics directly and ignoring the possibility of disturbances, we compute trajectories of the *embedding* system corresponding to each sampled control to over-approximate the reachable sets for *any* realizable disturbance. This allows us to robustly check the safety of the sampled control, as opposed to (10), which only considered the nominal trajectory.

Theorem 1. Let $\left[\underline{h}, \overline{h}\right]$ be any inclusion function for h, and consider the set

$$\mathcal{S}_{\mathcal{I}}^T(\mathbf{x}_{\text{init}}) := \Big\{ i \in \{1, \ldots, N\} \mid \inf_{t \in [0,T]} \underline{h}\left(\left[\underline{\Phi}, \overline{\Phi}\right]\left(t; \mathbf{x}_{\text{init}}; \mathbf{u}^{(i)}[\cdot]\right)\right) \geq 0 \Big\}. \tag{12}$$

All such sampled control sequences are safe controls:

$$\left\{ \mathbf{u}^{(i)}[\cdot] \mid i \in \mathcal{S}_{\mathcal{I}}^T(\mathbf{x}_{\text{init}}) \right\} \subseteq \mathcal{S}_{\mathbf{u}}^T(\mathbf{x}_{\text{init}}). \tag{13}$$

Proof. Consider any $\mathbf{u}^{(i)}[\cdot] \in \mathcal{S}_{\mathcal{I}}^T(\mathbf{x}_{\text{init}})$. Proposition 1 guarantees that its embedding system trajectory contains any

solution of (2) that could be realized by some disturbance signal. Therefore, by Definition 1 we have, for all $t \in [0, T]$,

$$\begin{aligned} 0 &\leq \underline{h}\left(\left[\underline{\Phi}, \overline{\Phi}\right]\left(t; \mathbf{x}_{\text{init}}; \mathbf{u}^{(i)}[\cdot]\right)\right) \\ &\leq h\left(\phi\left(t; \mathbf{x}_{\text{init}}; \mathbf{u}^{(i)}[\cdot], \mathbf{w}(\cdot)\right)\right), \end{aligned}$$

implying $\mathbf{u}^{(i)}[\cdot] \in \mathcal{S}_{\mathbf{u}}^{T}(\mathbf{x}_{\text{init}})$. □

Once the safety of a control signal has been verified by Theorem 1, it can be applied directly with the knowledge that no disturbance will ever cause a safety violation. This bypasses the need to account for disturbances with an "averaging step" (as in MPPI [5]) and motivates the use of the best sampled signal that is safe. Furthermore, the predicted interval trajectories give a much richer information structure to quantify which controls are best through a cost functional $J_{\text{int}} : L\left([0,T]; \mathbb{IR}^{d_{\mathbf{x}}}\right) \times L\left([0,T]; \mathbb{R}^{d_{\mathbf{u}}}\right) \to \mathbb{R}$. For example, given any inclusion function $\left[\underline{J}, \overline{J}\right]$, the choice

$$J_{\text{int}}\left(\left[\underline{\mathbf{x}}, \overline{\mathbf{x}}\right](\cdot), \mathbf{u}(\cdot)\right) := \overline{J}\left(\left[\underline{\mathbf{x}}, \overline{\mathbf{x}}\right](\cdot), [\mathbf{u}, \mathbf{u}](\cdot)\right), \tag{14}$$

$$i^* = \underset{i \in \mathcal{S}_{\mathcal{I}}^{T}(\mathbf{x}_{\text{init}})}{\arg\min}\, J_{\text{int}}\left(\left[\underline{\Phi}, \overline{\Phi}\right](\cdot; \mathbf{x}_{\text{init}}; \mathbf{u}[\cdot]), \mathbf{u}[\cdot]\right). \tag{15}$$

would optimize a guaranteed (possibly conservative) upper bound for the cost incurred by any disturbance realization.

A reachable-set aware planner can then be easily constructed by replacing (10) with (12) and (11) with (15) in Algorithm 1. The resulting logic is outlined in Algorithm 2.

Algorithm 2 Sampling-Based Planner (Reachable Sets)

Require: Reference control sequence $\mathbf{u}^*[\cdot]$

function ROLLOUTREACHSET($\mathbf{u}[\cdot]$, $\mathbf{x}_{\text{init}}$)
  return $\left[\underline{\Phi}, \overline{\Phi}\right](\cdot; \mathbf{x}_{\text{init}}; \mathbf{u}[\cdot])$ by Euler integration
function SELECTBESTREACHSET($\left[\underline{\mathbf{x}}, \overline{\mathbf{x}}\right]^{(1:N)}(\cdot)$, $\mathbf{u}^{(1:N)}[\cdot]$)
  $\mathcal{S}_{\mathcal{I}}^{T}(\mathbf{x}_{\text{init}}) \leftarrow$ guaranteed safe samples from (12)
  return Optimal sample index as in (15)

$\mathbf{u}^{(1:N)}[\cdot] \leftarrow$ SAMPLE($\mathbf{u}^*[\cdot]$)
$\left[\underline{\mathbf{x}}, \overline{\mathbf{x}}\right]^{(1:N)}(\cdot) \leftarrow$ ROLLOUTREACHSET($\mathbf{u}^{(i)}[\cdot]$, $\mathbf{x}_{\text{init}}$) for $\mathbf{u}^{(i)}[\cdot] \in \mathbf{u}^{(1:N)}[\cdot]$
$i^* \leftarrow$SELECTBESTREACHSET$\left(\left[\underline{\mathbf{x}}, \overline{\mathbf{x}}\right]^{(1:N)}(\cdot)\right.$, $\left.\mathbf{u}^{(1:N)}[\cdot]\right)$
Apply $\mathbf{u}^{(i^*)}$ [1], save $\mathbf{u}^{(i^*)}[\cdot]$

Remark 2. Though the theoretical modifications introduced by Algorithm 2 are simple, their practical implementation was only recently made possible by modern technological advances. The software toolbox `immrax` [16] leverages both interval reachability techniques designed for speed and scalability and the computational features of the JAX platform [19]. In particular, automatic parallelization over a modern GPU is critical for the real-time performance of Algorithm 2, allowing all sampled control sequences to be processed simultaneously.

Remark 3. The subroutine ROLLOUTREACHSET in Algorithm 2 requires observing the system's current state, which may not always be feasible. In this work, all hardware experiments were conducted in an environment with optitrack cameras that reported the state very precisely. However, it would also be easy to generalize the reachability-aware Algorithm 2 to account for observation error by e.g. initializing the embedding system (8) with an interval that contains the true state with high confidence, rather than a single point.

D. Limitations

The safety guarantee provided by Theorem 1 rules out many forms of failure caused by disturbances. However, practical limitations remain in applying this result. In particular, it is possible that the sampling process in Algorithm 2 does not find any safe control sequence, even if one exists. As only finitely many sequences are sampled, and since each sequence conservatively over-approximates the effect disturbances could have on the system state, there is no guarantee that a safe control is sampled and then proven safe. Furthermore, as a receding horizon approach, control sequences in $\mathcal{S}_{\mathbf{u}}^{T}(\mathbf{x}_{\text{init}})$ may induce system states from which safety violations cannot be prevented, as long as such violations would occur after the prediction horizon T. That is, Algorithm 2 does not have any recursive feasibility guarantee. These factors explain all safety violations observed during our experiments in Section IV-B, and are discussed further there.

Another potential failure mode is related to the method used to compute predicted trajectories, usually using numerical integration. If the integrator performs poorly and significantly diverges from the true trajectory, controls that generate unsafe trajectories may fail to be detected. Furthermore, evaluating the infimum in (10) and (12) may be difficult in practice, and can be heuristically substituted with checking every discretized timestep in the numerically integrated trajectory. If the discretizations are too large, trajectories that are unsafe for short periods of time might be falsely considered safe.

Finally, computing trajectories of the embedding system (8) requires formalizing both a bound on the possible disturbance magnitude and the precise mathematical mechanism by which they affect system dynamics. If the true disturbances are larger than expected, or they differ from their modeled influence on system state, the safe control set $\mathcal{S}_{\mathbf{u}}^{T}(\mathbf{x}_{\text{init}})$ no longer accurately represents which control sequences are safe to apply. Therefore, one must carefully choose a reasonable disturbance model before applying Algorithm 2.

IV. Numerical Simulations and Experiments

This section evaluates Algorithm 2 in simulation and on real-world hardware. We first define the vehicle dynamics and planning objectives for our test system (Section IV-A). Next, we benchmark the algorithm's performance and robustness in simulation, including a comparison against a state-of-the-art reachability-aware planner [20] (Sections IV-B and IV-C). Finally, we demonstrate the controller's real-world viability on a physical 1/28th scale racecar (Section IV-D).

A. Vehicle Dynamics and Objective

We evaluate our controller on a racecar with mass m and length ℓ_{f} (ℓ_{r}) from the center of mass to the front (rear) axle.

We use a dynamic bicycle model [23] with states x, y, ϕ, v_{long}, v_{lat}, and ω. These represent the 2D position, heading, longitudinal and lateral velocities, and yaw rate, respectively. The control inputs are steering u_{steer} and acceleration (throttle and braking) u_{acc}. Tire slip forces are modeled using the Pacejka Magic Formula [24]:

$$t\left(s\right) := D\sin(C + \arctan(Bs)), \tag{16}$$

$$F_{\text{f}} := \ell_{\text{r}}\frac{mg}{\ell_{\text{f}}+\ell_{\text{r}}}t\left(-\arctan\left(\frac{\omega\ell_{\text{f}}+v_{\text{lat}}}{v_{\text{long}}}\right)+u_{\text{steer}}\right), \tag{17}$$

$$F_{\text{r}} := \ell_{\text{f}}\frac{mg}{\ell_{\text{f}}+\ell_{\text{r}}}1.15t\left(\arctan\left(\frac{\omega\ell_{\text{r}}-v_{\text{lat}}}{v_{\text{long}}}\right)\right). \tag{18}$$

Input acceleration acts longitudinally on the vehicle according to a motor model:

$$m_{\text{acc}}\left(v_{\text{long}}, u_{\text{acc}}\right) := C_a\left(u_{\text{acc}} - \frac{v_{\text{long}}}{C_v} - C_e\right). \tag{19}$$

The complete system dynamics are therefore given by:

$$\begin{bmatrix}\dot{x}\\ \dot{y}\\ \dot{\phi}\\ \dot{v}_{\text{long}}\\ \dot{v}_{\text{lat}}\\ \dot{\omega}\end{bmatrix} = \begin{bmatrix} v_{\text{long}}\cos(\phi) - (v_{\text{lat}}+w_{v_{\text{lat}}})\sin(\phi)\\ v_{\text{long}}\sin(\phi) + (v_{\text{lat}}+w_{v_{\text{lat}}})\cos(\phi)\\ \omega + w_\phi\\ m_{\text{acc}}\left(v_{\text{long}}, u_{\text{acc}}\right)\\ m^{-1}(F_{\text{f}}+F_{\text{r}}-mv_{\text{long}}\omega)\\ I_z^{-1}(F_{\text{f}}\ell_{\text{f}}-F_{\text{r}}\ell_{\text{r}})\end{bmatrix}. \tag{20}$$

Here, $w_{v_{\text{lat}}} \in \left[\underline{w}_{v_{\text{lat}}}, \overline{w}_{v_{\text{lat}}}\right]$ and $w_\phi \in \left[\underline{w}_\phi, \overline{w}_\phi\right]$ are bounded, adversarial disturbances. These terms account for the inherent difficulty of accurately estimating tire slip and lateral velocity, a process that typically requires successive numerical differentiation of position data and is highly susceptible to sensor noise [25], [26]. Algorithm 2 robustly selects control actions that remain safe for any realization of $w_{v_{\text{lat}}}$ and w_ϕ.

For small v_{long}, the calculation of the slip forces F_{f} and F_{r} in (17)–(18) becomes singular. To ensure numerical stability, we switch to a simplified, kinematic bicycle model when longitudinal speed passes below a threshold. Given the *slip-slide* angle

$$\beta := \arctan\left(\frac{\ell_{\text{r}}}{\ell_{\text{f}}+\ell_{\text{r}}}\tan(u_{\text{steer}})\right), \tag{21}$$

the kinematic model predicts dynamics

$$\begin{bmatrix}\dot{x}\\ \dot{y}\\ \dot{\phi}\\ \dot{v}_{\text{long}}\\ \dot{v}_{\text{lat}}\\ \dot{\omega}\end{bmatrix} = \begin{bmatrix} v_{\text{long}}\cos(\phi+\beta)\\ v_{\text{long}}\sin(\phi+\beta) + w_{v_{\text{lat}}}\\ \frac{1}{\ell_{\text{f}}+\ell_{\text{r}}}v_{\text{long}}\cos(\beta)\tan(u_{\text{steer}}) + w_\phi\\ m_{\text{acc}}\left(v_{\text{long}}, u_{\text{acc}}\right)\\ -v_{\text{lat}}\\ -(\omega-\dot{\phi})\end{bmatrix}. \tag{22}$$

Here, $w_{v_{\text{lat}}}$ and w_ϕ are used similarly as in (20) to account for modeling errors.

Remark 4. There are several reasonable methods to extend the model switching motivated by the singularity in (17)–(18) to the interval-valued embedding system. Whenever the interval state does *not* overlap the v_{long} threshold, the selection between (20) and (22) is unambiguous. If there is an overlap, one option is to evaluate embedding systems for both dynamics separately and use the smallest interval containing the result of both. In practice, we have noticed that this is needlessly conservative, and simply using the largest value of v_{long} in the input interval is sufficient.

Despite using Cartesian coordinates for reachability computations, the safety and performance criteria are still most easily represented in the Frenet reference frame. By converting to coordinates progress s, lateral error e_{lat}, heading error e_ϕ, velocity components v_{long} and v_{lat}, and angular velocity error e_ω (all relative to a reference raceline), we can naturally specify notions such as "avoid collisions" and "drive quickly".

Assumption 3 (Track Observability)**.** We assume that during each control computation, the planner has access to the approaching track geometry up to time horizon T. Specifically, this information is represented as a reference raceline $s^*(\cdot)$, the lateral distance to the left and right track boundaries $[\underline{e}_{\text{lat}}, \overline{e}_{\text{lat}}]$, and desired longitudinal velocity v^*_{long}.

Assumption 3 is satisfied by the standard outputs of many common online observability and localization algorithms. In this work, and for simplicity of presentation, we use fixed environments where such information is readily available. Once the track geometry is known, specifying safe (non-colliding) states is simple in the Frenet frame:

$$h\left(\mathbf{x}\right) := -\operatorname{dist}\left(e_{\text{lat}}, \left[\underline{e}_{\text{lat}}, \overline{e}_{\text{lat}}\right]\left[s\right]\right). \tag{23}$$

Similarly, we design a cost function to incentivize staying near the raceline and progressing quickly along it. We have four running cost terms:

$$J_{\text{center}}\left(\mathbf{x}\left(t\right)\right) = \left|e_{\text{lat}}\left(t\right)\right|, \tag{24}$$

$$J_{\text{bnd}}\left(\mathbf{x}\left(t\right)\right) = \max\Big\{0, \arctan\big(-100\times \left(\operatorname{dist}(e_{\text{lat}}, \partial\left[\underline{e}_{\text{lat}}, \overline{e}_{\text{lat}}\right]\left[s\left(t\right)\right]) + 0.05\right)\big) + \frac{\pi}{2}\Big\}, \tag{25}$$

$$J_{\text{velR}}\left(\mathbf{x}\left(t\right)\right) = \left(v_{\text{long}}\left(t\right) - v^*_{\text{long}}\left[s\left(t\right)\right]\right)^2, \tag{26}$$

$$J_{\text{velN}}\left(\mathbf{x}\left(t\right)\right) = -\min\{v_{\text{long}}\left(t\right), 0\}, \tag{27}$$

and a single terminal cost $J_{\text{prog}}\left(\mathbf{x}(\cdot)\right) = s\left(0\right) - s\left(T\right)$. These terms are combined in a (positive) weighted sum to give the full cost function

$$J\left(\mathbf{x}(\cdot), \mathbf{u}(\cdot)\right) := \gamma_{\text{prog}}J_{\text{prog}}\left(\mathbf{x}(\cdot)\right) + \int_0^T \gamma_{\text{center}}J_{\text{center}}\left(\mathbf{x}\left(\tau\right)\right) + \gamma_{\text{bnd}}J_{\text{bnd}}\left(\mathbf{x}\left(\tau\right)\right) + \gamma_{\text{velR}}J_{\text{velR}}\left(\mathbf{x}\left(\tau\right)\right) + \gamma_{\text{velN}}J_{\text{velN}}\left(\mathbf{x}\left(\tau\right)\right)\,\mathrm{d}\tau. \tag{28}$$

Remark 5. When converting from Cartesian to Frenet coordinates, and at several points while evaluating (28), it is necessary to identify the desired velocity v^*_{long} or track boundary distances $[\underline{e}_{\text{lat}}, \overline{e}_{\text{lat}}]$ corresponding to the current progress s. This is easily accomplished for individual trajectories, but necessitates additional consideration for intervals. In principle, and with a closed-form expression for the observability information of Assumption 3, the inclusion function machinery of Section III-A is sufficient. For computational speed, and to more closely model how Assumption 3 may be satisfied

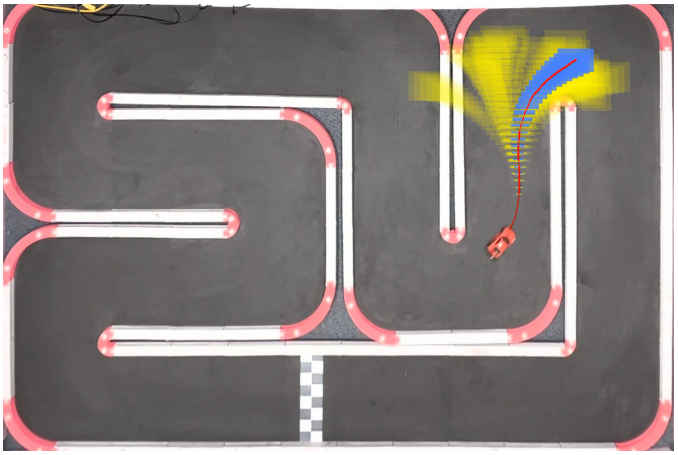


Fig. 1: Snapshot of Algorithm 2 being deployed on a 1/28th scale vehicle test platform. Computed over-approximations of the system's reachable sets are shown for a selection of the sampled controls, with the optimal control in blue and others in yellow. The centerpoint trajectory of the planned reachable sets is shown in red.

in practice, we provide track geometry as a discretized array of points and approximate the conversion of an interval by considering trajectories corresponding to each of its four vertices in the position plane separately and collecting the results. Since the reachable sets do not grow too large over the prediction horizon, if none of these trajectories result in collision, it is a strong indication that the safety check in (12) will be satisfied. We empirically validate this approach in the sequel.

B. Simulations

With the dynamics described in Section IV-A as a case study, we performed a simulated, parameter sweep comparison between the reachability-aware Algorithm 2 and the baseline, trajectory only Algorithm 1. Both controllers were tasked with driving three consecutive laps around the track shown in Figure 1 under artificially injected state disturbances. While Algorithm 1 operates without knowledge of these disturbances, Algorithm 2 is provided valid bounds on their maximum magnitude. Neither method was given the specific disturbance realizations *a priori*.

We conducted a parameter sweep comprising 30 initial states, 9 cost term weighting combinations, and 2 types of disturbance signals (540 total runs per controller). Trials were terminated early upon collision ("crash") or if the vehicle failed to advance for 10 consecutive seconds ("stall").

The reachability-based formulation demonstrated a marked improvement in safety. Algorithm 1 crashed 141 times, compared to only 26 for Algorithm 2. Notably, 25 of the 26 crashes under Algorithm 2 occurred when the vehicle was initialized precariously close to the track boundary and never sampled a safe control, strongly suggesting that the safe control set was empty ($\mathcal{S}^T_{\mathbf{u}}(\mathbf{x}_{\text{init}}) = \emptyset$). Excluding these unrecoverable initializations, incorporating reachability reduced the crash frequency by over 99.1%. We suspect that the last remaining crash was caused by a failure to guarantee recursive feasibility, as described in Section III-D.

In addition to greatly reducing crash frequency, Algorithm 2 displays greater robustness to parameter variation than Algorithm 1 (Table I). The success rate is largely independent of disturbance realization and cost function weights, de-coupling safety guarantee from performance tuning. Indeed, by robustly accounting for disturbances, the safety considerations of Algorithm 2 had an emergent, positive impact on performance, resulting in universally faster lap times.

C. Comparison

We conducted a bi-directional, simulation-based comparison against the state-of-the-art reachability enabled planner [20]. Our method achieves similar performance, maintains safety, and is scalable to larger system dimensions without any pre-computation.

In [20, Section VI], an HJ reachability-based method is used to navigate a racecar (modeled as a 3D Dubins vehicle) around a racetrack. We replicate this experiment in simulation using the same track geometry and system dynamics. Our algorithm achieves the same safety criteria (preventing all collisions) as the original work, and drives the vehicle at an average speed of 0.98 m/s. Compared to the baseline's reported 1.04 m/s, we incur a 6% speed penalty. As discussed in [20], the rich information stored in the HJ value function is used to *filter* every rollout, improving the sample efficiency of finding an optimal control and therefore performance overall. No such information is included in our interval reachable sets. However, computing this value function comes at a heavy cost as it must be evaluated and stored on a dense grid over the state space. On a system equipped with a Intel Xeon Gold 6230 CPU and NVIDIA Quadro RTX 8000 GPU, this took over 38 hours and the resulting file was 719.1 MB.

Such overhead grows exponentially in system dimension, making it difficult to apply the state-of-the-art method to our 6D model from Section IV-A. To compute the value function for the system and environment pictured in Figure 1, it was necessary to reduce the granularity of the evaluation grid, and even then, the computation took over 115 hours on the same high-powered system, and the solution occupied 2.4 GB. Moreover, due to imprecision in interpolating between the stored (coarse) evaluations of the value function on our 6D vehicle, the HJ-based controller crashed before completing the same 3-lap trial developed in Secion IV-B.

D. Experiment

Finally, we validated Algorithm 2 on actual hardware. For these experiments, we used the palm-sized, remote control vehicle platform Buzzracer developed in prior work [27]. The controller was executed on a computer equipped with an Intel i9-10920X CPU and NVIDIA RTX3090 GPU, and inputs were broadcast wirelessly to the vehicle over a local area network (LAN). Algorithm 2 was configured to sample $N = 1024$ control signals and predict $M = 30$ integration steps (of size $\Delta_{\mathbf{u}}t = 0.02$ seconds) for each sample every control loop.

By leveraging heavy parallelization with JAX [19], the control loop ran at nearly 60Hz. The vehicle completed over 35 consecutive laps on the track pictured in Figure 1 without

TABLE I: Outcome of the 1080-run sweep, broken down by sweep variable and reachable-set filter setting. Shaded rows are runs with the reachable-set filter enabled. Counts are runs with outcome `race_finished`; lap times are means over exactly those runs, and "—" marks cells with no successful runs. Each "Overall" cell aggregates 540 runs, each disturbance-direction cell 270 runs, and each centering-weight and velocity-scale cell 180 runs.

		Disturbance type		Centering weight γ_{center}			Velocity scale		
Reachable-set filter	Overall	Adversarial	Uniform	0.1	0.5	1.0	1.00	1.25	1.50
Successful runs									
Disabled	44	14	30	0	0	44	44	0	0
Enabled	504	247	257	166	168	170	171	172	161
Mean lap time (s)									
Disabled	5.97	6.02	5.94	—	—	5.97	5.97	—	—
Enabled	5.52	5.57	5.48	5.73	5.49	5.36	5.63	5.42	5.52

collision. These results validate both the disturbance structure chosen in (20) and Algorithm 2 itself as applicable to the real world.

V. Conclusion

Inspired by prior work using reachability as a control filter for sampling-based MPC, we leverage fast, interval-based methods to realize full reachable sets for each sample online and guarantee safety over a receding horizon. With this additional information, the proposed controller was able to greatly reduce safety violations while meeting performance objectives, both in simulation and on hardware. Future work may attempt to further strengthen our safety guarantees by developing a backup policy to preserve recursive feasibility in the event that no certifiably safe control is sampled. Alternatively, it could take advantage of the proposed online approach to address problems in dynamic environments or using adaptive control.

Acknowledgments

The authors wish to thank Dr. Javier Borquez for generously providing clarifying comments on and detailed experimental data regarding [20] that aided in development of the comparisons presented in Section IV-C.

References

[1] M. Schwenzer, M. Ay, T. Bergs, and D. Abel, "Review on model predictive control: an engineering perspective," *The International Journal of Advanced Manufacturing Technology*, vol. 117, no. 5, pp. 1327–1349, Nov. 1, 2021.

[2] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model Predictive Contouring Control for Time-Optimal Quadrotor Flight," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3340–3356, Dec. 2022.

[3] C. Mastalli et al., "Crocoddyl: An Efficient and Versatile Framework for Multi-Contact Optimal Control," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, May 2020, pp. 2536–2542.

[4] F. Li, H. Li, and C. Wu, "Gaussian Process Based Learning Model Predictive Control With Application to USV," *IEEE Transactions on Industrial Electronics*, vol. 71, no. 12, pp. 16 388–16 397, Dec. 2024.

[5] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving," *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, Dec. 2018.

[6] J. Yin, Z. Zhang, and P. Tsiotras, "Risk-Aware Model Predictive Path Integral Control Using Conditional Value-at-Risk," in *IEEE International Conference on Robotics and Automation (ICRA)*, May 2023, pp. 7937–7943.

[7] X. Zhang, J. Lu, Y. Hui, H. Shen, L. Xu, and B. Tian, "RAPA-Planner: Robust and Efficient Motion Planning for Quadrotors Based on Parallel RA-MPPI," *IEEE Transactions on Industrial Electronics*, vol. 72, no. 7, pp. 7149–7159, Jul. 2025.

[8] I. S. Mohamed, J. Xu, G. S. Sukhatme, and L. Liu, "Toward Efficient MPPI Trajectory Generation With Unscented Guidance: U-MPPI Control Strategy," *IEEE Transactions on Robotics*, vol. 41, pp. 1172–1192, 2025.

[9] F. Wang, Y. Cheng, and C. Tao, "MPPI-DBAS: Safe Trajectory Optimization With Adaptive Exploration," in *44th Chinese Control Conference (CCC)*, Jul. 2025, pp. 1616–1622.

[10] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control Barrier Functions: Theory and Applications," in *18th European Control Conference (ECC)*, Jun. 2019, pp. 3420–3431.

[11] J. Yin, C. Dawson, C. Fan, and P. Tsiotras, "Shield Model Predictive Path Integral: A Computationally Efficient Robust MPC Method Using Control Barrier Functions," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7106–7113, Nov. 2023.

[12] P. Mestres, Y. Chen, E. Dall'Anese, and J. Cortés, "Control Barrier Function-based Safety Filters: Characterization of Undesired Equilibria, Unbounded Trajectories, and Limit Cycles," *Journal of Nonlinear Science*, vol. 36, no. 2, p. 41, Apr. 13, 2026.

[13] S. M. Harwood and P. I. Barton, "Efficient polyhedral enclosures for the reachable set of nonlinear control systems," *Springer London*, Feb. 2016.

[14] M. Althoff, "An introduction to CORA 2015," in *ARCH14-15. 1st and 2nd international workshop on applied veRification for continuous and hybrid systems*, G. Frehse and M. Althoff, Eds., ser. EPiC series in computing, vol. 34, EasyChair, 2015, pp. 120–151.

[15] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-Jacobi reachability: A brief overview and recent advances," in *IEEE 56th Annual Conference on Decision and Control (CDC)*, Dec. 2017, pp. 2242–2253.

[16] A. Harapanahalli, S. Jafarpour, and S. Coogan, "immrax: a parallelizable and differentiable toolbox for interval analysis and mixed monotone reachability in JAX," *IFAC-PapersOnLine*, vol. 58, no. 11, pp. 75–80, 2024.

[17] B. Gould, A. Harapanahalli, and S. Coogan, "Automatic and Scalable Safety Verification Using Interval Reachability With Subspace Sampling," *IEEE Control Systems Letters*, vol. 9, pp. 1592–1597, 2025.

[18] A. Harapanahalli and S. Coogan, "Parametric Reachable Sets Via Controlled Dynamical Embeddings," in *IEEE 64th Conference on Decision and Control (CDC)*, Rio de Janeiro, Brazil: IEEE, Dec. 9, 2025, pp. 2694–2700.

[19] J. Bradbury et al., *JAX: Composable Transformations of Python+NumPy Programs*, version 0.3.13, 2018.

[20] J. Borquez, L. Raus, Y. U. Ciftci, and S. Bansal, "DualGuard MPPI: Safe and Performant Optimal Control by Combining Sampling-Based MPC and Hamilton-Jacobi Reachability," *IEEE Robotics and Automation Letters*, vol. 10, no. 7, pp. 6944–6951, Jul. 2025.

[21] K. Shen and J. Scott, "Rapid and Accurate Reachability Analysis for Nonlinear Dynamic Systems by Exploiting Model Redundancy," *Computers & Chemical Engineering*, vol. 106, Aug. 1, 2017.

[22] L. Jaulin, M. Kieffer, O. Didrit, and É. Walter, *Applied Interval Analysis*. London: Springer London, 2001.

[23] R. Rajamani, *Vehicle Dynamics and Control* (Mechanical Engineering Series). Boston, MA: Springer US, 2012.

[24] H. B. Pacejka and E. Bakker, "The Magic Formula Tyre Model," *Vehicle System Dynamics*, vol. 21, pp. 1–18, sup001 Jan. 1992.

[25] Y.-W. Liao and F. Borrelli, "An Adaptive Approach to Real-Time Estimation of Vehicle Sideslip, Road Bank Angles, and Sensor Bias," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 8, pp. 7443–7454, 2019.

[26] A. Y. Ungoren, H. Peng, and H. E. Tseng, "A Study on Lateral Speed Estimation Methods," *International Journal of Vehicle Autonomous Systems*, vol. 2, no. 1–2, pp. 126–144, 2004.

[27] Z. Zhang and P. Tsiotras, "BuzzRacer: A Palm-sized Autonomous Vehicle Platform for Testing Multi-Agent Adversarial Decision-Making," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2024, pp. 8510–8515.